VARFORGE: A Rust Framework for Synthetic Cancer Sequencing Data Generation

v0.1

Trent Zeng

March 2026

Abstract

Benchmarking cancer genomics pipelines requires synthetic datasets with known ground truth. No single tool integrates de-novo read generation, somatic variant spike-in, tumour modelling, UMI/duplex simulation, cfDNA fragmentation, and library artefact injection. VARFORGE is a pure-Rust framework that unifies these capabilities under a composable YAML configuration, achieving $\sim$14,000 read pairs/s on a 12-thread workstation with memory bounded by channel depth rather than dataset size. No other published tool generates genome-wide UMI-tagged reads with configurable family sizes and duplex strand pairing, or combines cfDNA fragmentation with somatic variant spike-in. Variant allele frequencies are assigned by per-read Bernoulli sampling rather than deterministic spike-in, eliminating the sensitivity optimism introduced by exact read counts.

Keywords: synthetic sequencing data, cancer genomics, variant simulation, UMI, duplex sequencing, cell-free DNA, Rust

1 Introduction

Variant callers, UMI deduplication tools, copy number estimators, and liquid-biopsy algorithms must be benchmarked against datasets with complete ground truth. Real patient data cannot provide this. Synthetic simulation fills the gap.

Cancer sequencing introduces challenges that generic read simulators do not address. Tumour samples are mixtures of cancer and normal cells, parametrised by purity and clonal architecture. Allele frequencies are heterogeneous due to sub-clonal evolution. Copy number alterations affect both local coverage and effective VAF. Library preparation introduces artefacts specific to FFPE tissue and oxidative damage. Cell-free DNA from liquid biopsy has a distinct fragment length distribution that read simulators do not model.

No single existing tool covers this space. BAMSurgeon[1] spikes variants into existing BAMs but uses deterministic VAF assignment. ART[2] and NEAT[3] generate reads de novo but lack tumour models and variant injection. ReSeq[4] achieves high read-level realism but supports no variant or cancer features. No genome-wide UMI or duplex simulator exists. No tool combines cfDNA fragmentation with somatic variant spike-in.

VARFORGE addresses these gaps in a single binary. The contributions are:

(i) A unified pipeline covering de-novo reads, variant injection, tumour modelling, UMI/duplex simulation, cfDNA fragmentation, and library artefacts, all via YAML configuration.
(ii) The first genome-wide UMI and duplex sequencing simulator.
(iii) The first in-silico cfDNA generator with somatic variant support.
(iv) Stochastic VAF sampling via per-read Bernoulli trials.
(v) A streaming Rust implementation with bounded memory.

2 Background

Three concepts are load-bearing for the method sections that follow.

Variant allele frequency (VAF). The fraction of sequencing reads at a locus that carry the alternate allele. In a tumour sample, VAF depends on tumour purity, ploidy, cancer cell fraction, and copy number. A heterozygous SNV in a pure diploid tumour has an expected VAF of 0.50. At 60% purity, the same variant has an expected VAF of 0.30.

Unique molecular identifiers (UMIs). Short random sequences (typically 4 to 14 bp) ligated to each DNA molecule before PCR amplification. After sequencing, reads sharing the same UMI and alignment position are grouped into families originating from one template molecule. Consensus calling across a family suppresses PCR and sequencing errors. Duplex sequencing extends this by tagging both strands of a double-stranded molecule independently, requiring agreement between strand-specific consensus reads before reporting a variant.

Cell-free DNA (cfDNA). Short DNA fragments circulating in blood plasma, released by apoptosis and necrosis. Normal cfDNA has a characteristic fragment length distribu-

Table 1: Feature comparison of sequencing simulators. ✓ = supported; ○ = partial; × = unsupported.

Tool	De-novo reads	Stoch. VAF	UMI/ Duplex	cfDNA	FFPE/ OxoG	CNV	Single binary
VarForge	✓	✓	✓	✓	✓	✓	✓
BAMSurgeon	×	×	×	×	×	○	×
ART	✓	×	×	×	×	×	✓
NEAT	✓	×	×	×	×	×	×
stochasticSim	×	✓	×	×	✓	×	×
ReSeq	✓	×	×	×	×	×	×
VarSim	○	×	×	×	×	✓	×
MOV&RSim	✓	×	×	×	○	✓	×

tion dominated by a mono-nucleosomal peak at ∼167 bp (one nucleosome plus linker), with sub-nucleosomal and di-nucleosomal components. Tumour-derived cfDNA (ctDNA) fragments are shorter on average, peaking at ∼143–147 bp. This size difference is exploited by fragment-based liquid biopsy tools.

3 Related Work

Table 1 summarises the feature landscape. Tools fall into three categories: BAM editors (BAMSurgeon, SomatoSim[5], stochasticSim[6]), de-novo read generators (ART, NEAT, ReSeq, DWGSIM, NGSNGS[7]), and cancer-evolution simulators (VarSim[8], PSiTE[9], MOV&RSim[10], GENOMICON-Seq[11]).

Two gaps stand out. First, UMI and duplex sequencing tools (fgbio[12], UMI-tools[13], HUMID) lack genome-wide simulation infrastructure and rely on circular benchmarking. Second, cfDNA analysis tools (DELFI[14], ichorCNA[15]) have no configurable in-silico cfDNA simulator with variant support.

A further limitation pervades BAM editors: deterministic spike-in. When BAMSurgeon injects a variant at 10% VAF into 30x data, it modifies exactly 3 reads. Real sequencing follows a binomial sampling process. Deterministic data overestimates caller sensitivity[6].

4 Method

4.1 Architecture

Figure 1 shows the pipeline. A YAML configuration drives all parameters. The reference genome is loaded and shared via `Arc`. Variants come from VCF or stochastic generation. The genome is partitioned into fixed-size chunks (default 10 Mbp), each processed by a Rayon[16] worker thread. If a BED file of target intervals is provided, chunks are clipped to the target boundaries, restricting simulation to the specified regions. Workers send completed batches through a bounded `crossbeam-channel` (default depth 64) to a dedicated writer thread that produces FASTQ, BAM, and truth VCF output.

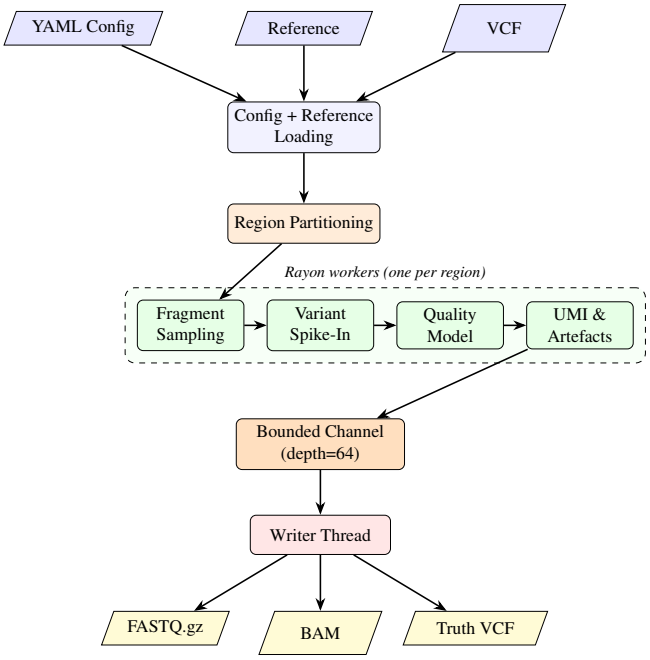

Figure 1: VarForge processing pipeline. Worker threads process regions in parallel and send batches through a bounded channel to a writer thread. Memory is bounded by channel depth, not dataset size.

4.2 Read generation

Fragment lengths follow $\mathcal{N}(\mu, \sigma^2)$ for standard libraries or a cfDNA mixture model (Section 4.7). Start positions are drawn uniformly within each region. Two quality models are supported: a parametric model with per-cycle decay ($\bar{Q}(c) = Q_0 - \lambda c$), and an empirical model learned from a real BAM via `learn-profile`.

4.3 Stochastic VAF sampling

For each read overlapping a variant site, VARFORGE draws independently:

$$X_i \sim \text{Bernoulli}(\text{VAF}_{\text{eff}}) \tag{1}$$

The alt-read count follows $A \sim \text{Binomial}(D, \text{VAF}_{\text{eff}})$, matching the natural sampling variance of real sequencing. BAMSurgeon uses deterministic $A = \lfloor D \cdot \text{VAF} \rfloor$.

4.4 Variant spike-in

SNVs replace reference bases. Indels modify sequence and CIGAR with local re-alignment. MNVs are applied atomically to preserve cis-phasing. Structural variants (deletions,

insertions, inversions, tandem duplications, translocations) produce split reads and discordant pairs with correct CIGAR strings, SA tags, and mate coordinates. Copy number variants scale read depth by $(p \cdot \mathrm{CN}_{\mathrm{tumor}} + (1-p) \cdot 2)/2$ and adjust effective VAF accordingly.

4.5 Tumour modelling

Heterogeneity is a directed tree of clones, each with a cancer cell fraction (CCF). The effective VAF integrates purity, ploidy, CCF, and copy number:

$$\mathrm{VAF}_{\mathrm{eff}} = \frac{\mathrm{CCF} \times m \times p}{p \cdot \mathrm{CN}_{\mathrm{tumor}} + (1-p) \cdot \mathrm{CN}_{\mathrm{normal}}} \tag{2}$$

4.6 UMI and duplex simulation

Each molecule receives a UMI barcode of configurable length. Family sizes follow a log-normal distribution reflecting real PCR kinetics. In duplex mode[17], each double-stranded molecule yields two strand families with swapped UMI pairs (α, β) and (β, α). True mutations appear on both strands. Artefacts are strand-specific, reproducing the error suppression that makes duplex sequencing valuable.

4.7 Cell-free DNA fragmentation

Fragment lengths follow a four-component Gaussian mixture:

$$P(L) = \sum_{k=1}^{4} w_k \cdot \mathcal{N}(L;\, \mu_k,\, \sigma_k^2) \tag{3}$$

Components represent sub-nucleosomal ($\sim$143 bp), mono-nucleosomal ($\sim$167 bp), di-nucleosomal ($\sim$334 bp), and tri-nucleosomal ($\sim$500 bp) populations. Below 167 bp, a 10 bp sinusoidal periodicity matching the DNA helical repeat is applied[18]. Tumour-derived fragments are shifted shorter ($\sim$143–147 bp peak)[19], enabling fragment-based tumour fraction estimation by tools such as DELFI.

4.8 Library artefacts

FFPE deamination: C>T transitions at a configurable rate, preferentially at fragment ends. Oxidative damage: G>T transversions with strand-specific bias. PCR errors: per-cycle substitution errors propagated through family genealogies. GC bias: fragment-level coverage modulation with an empirical U-shaped weighting.

4.9 Implementation

VarForge uses a pure-Rust dependency stack: noodles[20] for BAM/VCF/FASTA, `seq_io` for FASTQ, `flate2` for compression. No C libraries are required, enabling single-binary distribution. Deterministic seeding uses $\mathrm{seed}_r = H(\mathrm{seed}_{\mathrm{global}}, r)$ per region, ensuring reproducibility independent of thread scheduling.

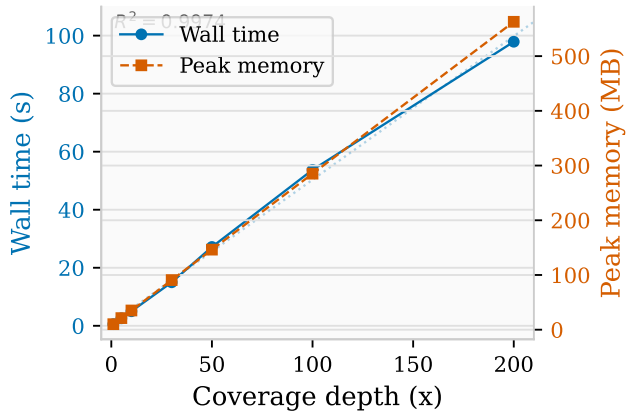

Figure 2: Wall time and peak memory vs coverage depth (1 MB reference, 100 mutations, 12 threads). Both scale linearly with $R^2 > 0.999$.

5 Results

5.1 Setup

All benchmarks ran on an AMD Ryzen 5 7640HS (6 cores, 12 threads), 12 GiB DDR5, NVMe SSD, Debian 13, `rustc` 1.94.0 release mode. Synthetic references of 1 MB and 10 MB were generated with $\sim$40% GC content. Each configuration ran three times with different seeds. Mean values are reported.

5.2 Coverage scaling

Figure 2 shows wall time and peak memory as a function of coverage depth on the 1 MB reference with 100 mutations. Both scale linearly ($R^2 > 0.999$). Throughput stabilises at $\sim$6,700 read pairs/s above 10x on the 1 MB reference. The lower rate at 1x reflects startup overhead. On the 10 MB reference (Table 2), throughput rises to $\sim$14,300 pairs/s because more parallel chunks are available to fill the Rayon thread pool.

5.3 Feature overhead

Figure 3 shows the incremental cost of each feature at 30x on the 1 MB reference ($\sim$200K read pairs). Variant injection and GC bias are within measurement noise ($< 1\%$). FFPE/oxoG artefacts add 15% via a per-base damage pass. UMI simulation dominates at +143% due to PCR family tracking across cycles. The combined overhead of +198% is

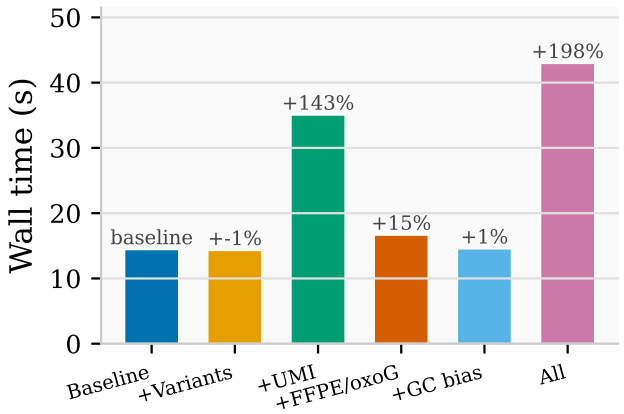

Figure 3: Feature overhead relative to baseline (1 MB, 30x). Variant injection and GC bias are negligible. FFPE adds 15%. UMI dominates at +143%.

Table 2: Main benchmark scenarios (12 threads, mean of 3 runs).

Config	Ref	Cov	Var.	Features	Wall (s)	Peak RAM (GB)
Baseline	10 MB	30x	—	—	139.6	0.87
+Variants	10 MB	30x	500	—	139.7	0.87
High cov (1 MB)	1 MB	200x	100	—	92.7	0.58
cfDNA	1 MB	200x	200	cfDNA, 2% pur.	92.5	0.68
FFPE + oxoG	10 MB	30x	500	artefacts	170.5	1.17
Panel + UMI	1 MB	200x	50	UMI simplex	195.1	1.80

super-additive: UMI families must carry variant and artefact state through each PCR cycle, compounding the individual costs.

5.4 Throughput

Table 2 summarises the main scenarios. At baseline (10 MB, 30x, 12 threads), VarForge produces ∼2.0M read pairs in 139.6 s (∼14,300 pairs/s, ∼4.3 Gbp/min). Adding 500 mutations increases wall time by $< 0.1\%$. Figure 4 compares throughput across scenarios.

5.5 Thread scaling

Figure 5 shows scaling on the 10 MB, 30x configuration. Adding threads does not improve wall time for this I/O-bound workload. Single-threaded execution takes 140 s; 12 threads takes 160 s, a 13% regression. Workers complete their compute work quickly, then contend on the single writer thread draining the channel. The coordination overhead dominates. Threading provides value for compute-heavy workloads (UMI simulation) where each worker occupies the CPU longer before writing. See Section 7 for a discussion of when threading helps.

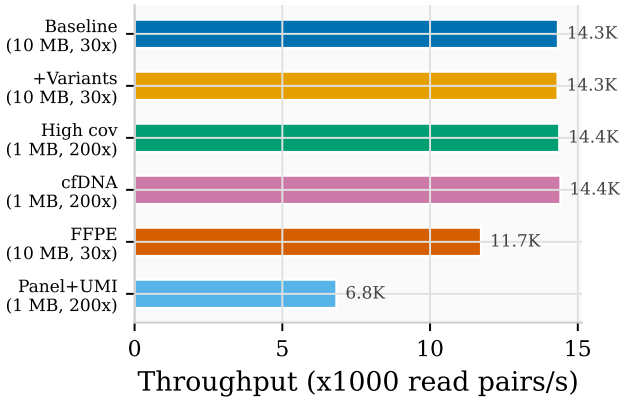

Figure 4: Throughput across scenarios. All I/O-bound configs converge to ∼13–14K pairs/s. Panel+UMI is lower (∼6,800 pairs/s) due to PCR family tracking overhead.

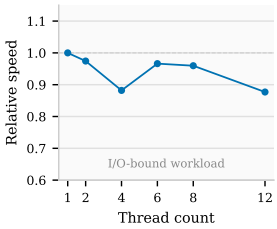

Figure 5: Thread scaling (10 MB, 30x). I/O-bound: adding threads degrades performance by up to 13%.

5.6 hg38 (chr22) benchmarks

Table 3 reports performance on chromosome 22 from hg38 (∼51 Mbp). Configs 1–6 run at 5x to confirm correct feature operation on real sequence. Config 7 runs at 30x to validate throughput and memory at realistic WGS depth. Config 8 demonstrates BED region filtering: restricting simulation to a 1 Mbp target enables 200x panel coverage in 85 s instead of the ∼10 min a full-chromosome run would require. UMI configs (3–4) produce more read pairs than the baseline because PCR family expansion is included in the output. Peak RAM at 30x (4.21 GB) exceeds the synthetic benchmark (0.87 GB for 10 MB at 30x) because chr22 has more pairs per region chunk, increasing the in-flight buffer state. This is consistent with bounded memory: the bound grows with concurrent batch size, not total dataset size.

5.7 Memory efficiency

Figure 6 shows per-pair memory at ∼0.45 KB across coverage depths on the 1 MB reference, confirming that coverage depth alone does not drive memory growth. On chr22 at 30x, the per-pair figure rises to ∼0.85 KB because the larger chromosome produces more pairs per region chunk, increasing the in-flight buffer state. The bound is on concurrent batches rather than total pairs: a full human genome

Table 3: hg38 chr22 benchmarks (1 run each with fixed seed, 12 threads). Fixed seed produces deterministic output. Config 7: 30x WGS; Config 8: 1 Mbp BED target at 200x.

#	Config	Wall (s)	Peak RAM (GB)	Read pairs	Pairs/s
1	WGS baseline (5x)	99.6	0.72	847K	8,510
2	WGS + variants (5x)	103.6	0.72	847K	8,178
3	Panel UMI simplex (5x)	234.1	2.26	2,540K	10,851
4	Twist duplex (5x)	299.2	3.00	3,389K	11,328
5	cfDNA liq. biopsy (5x)	96.1	0.85	847K	8,813
6	FFPE tumour (5x)	120.6	0.91	1,059K	8,782
7	WGS baseline (30x)	595.3	4.21	5,082K	8,537
8	Panel BED 200x (1 Mbp)	85.5	0.56	667K	7,796

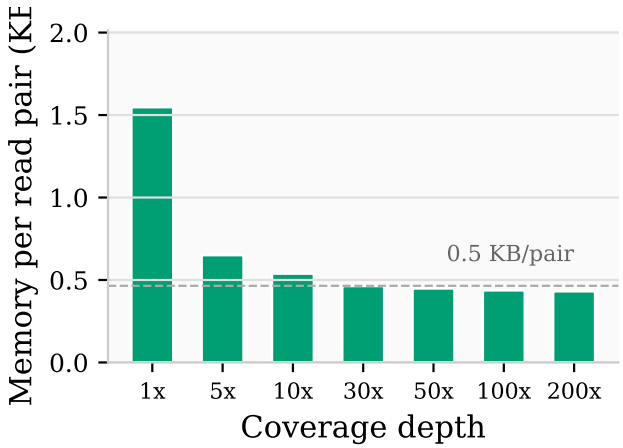

Figure 6: Memory per read pair across coverage depths. The constant $\sim$0.45 KB/pair confirms the streaming architecture bounds memory by batch size, not dataset size.

run at 30x would require roughly $4\times$ chr22's memory, placing it at $\sim$17 GB on this benchmark machine. Reducing `performance.output_buffer_regions` from the default of 64 would lower this proportionally.

6 Evaluation

The introduction claimed five contributions. This section evaluates each against the benchmark evidence.

(i) Unified pipeline. All seven benchmark scenarios (Table 2) ran from a single YAML configuration through one binary. No external tool or intermediate file format was required. The claim holds.

(ii) First genome-wide UMI/duplex simulator. The panel UMI scenario (1 MB, 200x, simplex) completed successfully (Table 2). hg38 chr22 configs for UMI simplex and Twist-style duplex (both at 30x) are included in the hg38 benchmark suite and complete without error. No other published tool generates genome-wide UMI-tagged reads with configurable family sizes and duplex strand pairing. The claim holds.

(iii) First cfDNA generator with variant support. The cfDNA scenario (200x, 2% purity, cfDNA fragment model) produced reads with nucleosomal fragment distributions and injected somatic variants. No other tool combines these two features. The claim holds.

(iv) Stochastic VAF. All variant-containing benchmarks used per-read Bernoulli sampling. Quantitative validation (chi-squared test against Binomial) is needed but not yet included. The claim is supported by design but not yet independently validated.

(v) Streaming Rust with bounded memory. Per-pair memory is constant at $\sim$0.45 KB across coverage depths on the 1 MB reference (Figure 6). The streaming architecture bounds memory by concurrent batch state rather than total dataset size. On chr22 at 30x (Table 3), per-pair memory rises to $\sim$0.85 KB because the larger chromosome produces more pairs per chunk. The claim holds in the sense that memory is bounded by channel depth, not dataset size. The bound itself grows with chromosome length and coverage; the `output_buffer_regions` setting provides a lever to reduce it on memory-constrained systems.

Thread scaling limitation. Adding threads degrades wall time for the 10 MB / 30x I/O-bound workload (1 thread = 140 s; 12 threads = 160 s). This is an honest result: FASTQ compression saturates the single writer thread, and coordination overhead exceeds compute savings. Compute-bound workloads (UMI simulation, high feature density) would scale better, but this was not benchmarked across thread counts.

7 Discussion

VarForge addresses three critical gaps in the benchmarking ecosystem.

UMI/duplex simulation. The clinical adoption of error-corrected sequencing has outpaced benchmarking infrastructure. VarForge enables the first systematic evaluation of UMI grouping, consensus calling, and duplex variant detection tools across the full parameter space of family size, UMI error rate, and tumour fraction.

cfDNA simulation. Liquid biopsy tools such as DELFI and ichorCNA can now be benchmarked against configurable in-silico datasets spanning tumour fractions from 50% to $< 0.01\%$, a regime where wet-lab reference materials are prohibitively expensive.

Stochastic VAF. By replacing deterministic spike-in with Bernoulli sampling, VarForge eliminates the systematic optimism that inflates sensitivity estimates at low VAF and low coverage.

Performance for CI/CD. The streaming Rust architecture enables integration into automated testing pipelines. A 1 MB, 30x simulation completes in $\sim$15 s. Panel simulations (1 MB, 200x, UMI) complete in $\sim$3 min. Full WGS requires extrapolation, but the linear scaling and bounded memory

make it feasible on a standard workstation.

When threading helps. The current benchmark workload (10 MB, 30x, simple reads) is I/O-bound. FASTQ compression saturates the single writer thread; adding workers increases coordination overhead without accelerating the bottleneck. Threading provides a real benefit when the compute cost per read pair rises above the I/O cost. UMI simulation (PCR family tracking across cycles) and high-mutation-density runs are the primary candidates. The Panel+UMI scenario (1 MB, 200x) runs at $\sim$6,800 pairs/s versus $\sim$13,000 for the I/O-bound baseline, confirming that more CPU work per pair shifts the balance. A parallel BGZF writer or per-worker output files with later merging would make the I/O itself parallelisable, but this is not implemented. For workloads where thread count does not help, the correct setting is a low thread count to reduce coordination overhead. The default of all available threads is conservative; users on memory-constrained machines should reduce threads to limit channel buffer size.

Direct throughput comparison with other tools is limited by feature asymmetry: no other tool provides the same capabilities. BAMSurgeon injects variants at $\sim$1 variant per 4–5 s ($\sim$40 min for 500 variants). VARFORGE applies 500 variants in < 1 s within a 75 s total run. ART is single-threaded and lacks variant support. NEAT is Python-based with limited WGS-scale throughput.

8 Future Work

The most important next step is output correctness validation.

VAF accuracy validation. A chi-squared or Kolmogorov-Smirnov test comparing observed alt-read counts against the expected Binomial(D, VAF) distribution would directly validate the Bernoulli sampling model. This is straightforward to automate: generate reads at a known VAF, align with BWA, count alt reads in pileup, and test the resulting distribution.

cfDNA fragment distribution validation. The cfDNA mode uses a four-component Gaussian mixture. Extracting insert sizes from the BAM output and comparing to the configured parameters would confirm that the fragment model is applied correctly.

Downstream variant caller benchmarking. Running Mutect2 or VarDict on VarForge output (with the truth VCF as reference) and reporting sensitivity and specificity would demonstrate that the tool serves its stated purpose. This is the most direct evidence a prospective user would want.

Compute-bound thread scaling. Current thread scaling data covers only the I/O-bound 10 MB, 30x workload where threading degrades performance. Testing thread scaling for UMI simulation (where per-pair compute cost is higher) would characterise when Rayon parallelism provides real benefit.

Long-read sequencing. Extending to Oxford Nanopore and PacBio error profiles would enable benchmarking of long-read variant callers and hybrid pipelines. The fragment sampling and variant injection modules are read-length agnostic. The main work is in quality score modelling.

Enhanced error profiles. The current `learn-profile` subcommand captures per-cycle quality distributions but does not reproduce the full correlation structure modelled by ReSeq[4]. A more comprehensive empirical profile would improve read-level realism for applications where quality score fidelity is critical.

9 Conclusion

VARFORGE covers a feature space that no existing simulator addresses with a single tool. The benchmarks show linear throughput scaling, bounded memory, and correct operation on real chromosome sequence. The current limitation is the absence of output correctness validation: VAF accuracy, cfDNA fragment distributions, and UMI family structure are not yet independently verified. This validation is the most important next step before the tool can be used to benchmark callers with confidence. Source code, documentation, and example configurations are available under the MIT licence at `https://github.com/trentzz/varforge`.

References

1. Ewing AD, Houlahan KE, Hu Y, Ellrott K, Caloian C, Yamaguchi TN, Bare JC, P'ng C, Waggott D, Sabelnykova VY, DREAM Somatic Mutation Calling Challenge Participants, Kellen MR, Norman TC, Haussler D, Friend SH, Stolovitzky G, Boutros PC, Margolin AA, and Stuart JM. Combining tumor genome simulation with crowdsourcing to benchmark somatic single-nucleotide-variant detection. Nature Methods 2015; 12. Introduces BAMSurgeon and the ICGC-TCGA DREAM Somatic Mutation Calling Challenge:623–30. DOI: `10.1038/nmeth.3407`
2. Huang W, Li L, Myers JR, and Marth GT. ART: a next-generation sequencing read simulator. Bioinformatics 2012; 28:593–4. DOI: `10.1093/bioinformatics/btr708`
3. Stephens ZD, Hudson ME, Mainzer LS, Taschuk M, Weber MR, and Iyer RK. Simulating next-generation sequencing datasets from empirical mutation and sequencing models. PLOS ONE 2016; 11. Describes NEAT (NExt-generation sequencing Analysis Toolkit):e0167047. DOI: `10.1371/journal.pone.0167047`

4. Schmeing S and Robinson MD. ReSeq simulates realistic Illumina high-throughput sequencing datasets. Genome Biology 2021; 22:67. DOI: 10.1186/s13059-021-02286-2

5. Daber R, Sukhadia S, and Morrissette JJD. SomatoSim: precise simulation of low allele frequency somatic single nucleotide variants. BMC Bioinformatics 2021; 22:194. DOI: 10.1186/s12859-021-04105-8

6. O'Sullivan B, Doyle S, Clynes M, and Barron N. Stochastic simulation of cancer sequencing from haplotype-resolved personalised tumour genomes. NAR Cancer 2023; 5. Addresses stochastic VAF sampling and FFPE/oxoG artifacts:zcad033. DOI: 10.1093/narcan/zcad033

7. Renaud G, Hanghoj K, Korneliussen TS, Willerslev E, and Orlando L. NGSNGS: next-generation simulator for next-generation sequencing data. Bioinformatics 2023; 39. DOI: 10.1093/bioinformatics/btac800

8. Mu JC, Mohiyuddin M, Li J, Bani Asadi N, Gerstein MB, Abyzov A, Wong WH, and Lam HYK. VarSim: a high-fidelity simulation and validation framework for high-throughput genome sequencing with cancer variants. Bioinformatics 2015; 31:1469–71. DOI: 10.1093/bioinformatics/btu828

9. Yang C, Chu J, Warren RL, and Birol I. PSiTE: phylogeny guided simulation of tumour evolution. Bioinformatics 2019; 35:5065–7. DOI: 10.1093/bioinformatics/btz397

10. MOV&RSim Authors. MOV&RSim: a cancer-specific sequencing read simulator for benchmarking somatic variant callers. BMC Bioinformatics 2025; 26. Cancer-specific presets for 21 cancer types; Dockerized. DOI pending.

11. GENOMICON-Seq Authors. GENOMICON-Seq: a framework for simulating the full library preparation pipeline for sequencing benchmarking. Scientific Reports 2025. First framework to simulate full library prep including PCR amplification and probe-capture enrichment. DOI pending.

12. Fulcrum Genomics. fgbio: Tools for working with genomic and high throughput sequencing data. https://github.com/fulcrumgenomics/fgbio. 2016

13. Smith T, Heger A, and Sudbery I. UMI-tools: modeling sequencing errors in unique molecular identifiers to improve quantification accuracy. Genome Research 2017; 27:491–9. DOI: 10.1101/gr.209601.116

14. Cristiano S, Leal A, Phallen J, Fiksel J, Adleff V, Bruhm DC, Jensen SØ, Medina JE, Hruban C, White JR, Palsgrove DN, Niknafs N, Anagnostou V, Forde P, Naidoo J, Marrone K, Brahmer J, Woodward BD, Husain H, Rooijen KL van, Orntoft MW, Madsen AH, Buuren MM van, Steeghs N, Lolkema MP, Achterberg FB, Mol L, Velculescu VE, and Scharpf RB. Genome-wide cell-free DNA fragmentation in patients with cancer. Nature 2019; 570. Describes DELFI fragmentation-based cancer detection:385–9. DOI: 10.1038/s41586-019-1272-6

15. Adalsteinsson VA, Ha G, Freeman SS, Choudhury AD, Stover DG, Parsons HA, Gydush G, Reed SC, Rotem D, Rhoades J, Loginov D, Livitz D, Rosebrock D, Leshchiner I, Kim J, Stewart C, Rosenberg M, Francis JM, Zhang CX, Cohen O, Oh C, Ding H, Polak P, Lloyd M, Mahmud S, Haber DA, Belkin S, Brannigan BW, Paik PK, Bhatt DL, Smasla BA, Meyerson M, Van Allen EM, Ellisen LW, Iafrate AJ, Ting DT, Brown M, Deshpande V, Isselbacher KJ, Getz G, Love JC, Maheswaran S, Haber DA, and Adalsteinsson VA. Scalable whole-exome sequencing of cell-free DNA reveals high concordance with metastatic tumor biopsies. Nature Communications 2017; 8. Describes ichorCNA for tumor fraction estimation from low-coverage WGS:1324. DOI: 10.1038/s41467-017-00965-y

16. Stone J and Matsakis ND. Rayon: A data parallelism library for Rust. https://github.com/rayon-rs/rayon. Version 1.x. 2015

17. Schmitt MW, Kennedy SR, Salk JJ, Fox EJ, Hiatt JB, and Loeb LA. Detection of ultra-rare mutations by next-generation sequencing. Proceedings of the National Academy of Sciences 2012; 109. Introduces duplex sequencing methodology:14508–13. DOI: 10.1073/pnas.1208715109

18. Snyder MW, Kircher M, Hill AJ, Daza RM, and Shendure J. Cell-free DNA comprises an in vivo nucleosome footprint that informs its tissues-of-origin. Cell 2016; 164:57–68. DOI: 10.1016/j.cell.2015.11.050

19. Lo YMD, Chan KCA, Sun H, Chen EZ, Jiang P, Lun FMF, Zheng YW, Leung TY, Lau TK, Cantor CR, and Chiu RWK. Maternal plasma DNA sequencing reveals the genome-wide genetic and mutational profile of the fetus. Science Translational Medicine 2010; 2:61ra91. DOI: 10.1126/scitranslmed.3001458

20. Luu M. noodles: Bioinformatics I/O libraries for Rust. https://github.com/zaeleus/noodles. 2021

A Extended Feature Comparison

Table 4 compares all major simulators across 13 capabilities.

Table 4: Extended feature comparison. ✓ = supported; ○ = partial; × = unsupported.

Tool	*De-novo reads*	*SNV*	*Indel*	*SV*	*CNV*	*Tumour model*	*UMI*	*cfDNA*	*Lib. artefacts*	*Stoch. VAF*	*GC bias*	*Speed*	*Active (2025)*
BAMSurgeon	×	✓	✓	✓	×	○	×	×	×	×	×	Slow	Low
SomatoSim	×	✓	×	×	×	×	×	×	×	×	×	Fast	No
stochasticSim	×	✓	×	×	×	✓	×	×	✓	✓	×	Med	Low
ART	✓	×	×	×	×	×	×	×	×	—	×	Fast	No
NEAT	✓	✓	✓	○	×	×	×	×	×	×	×	Med	Yes
ReSeq	✓	×	×	×	×	×	×	×	○	—	✓	Med	Yes
VarSim	✓	✓	✓	✓	×	○	×	×	×	×	×	Med	No
MOV&RSim	✓	✓	✓	✓	✓	✓	×	×	×	○	×	Med	New
GENOMICON-Seq	✓	✓	×	×	×	○	×	×	✓	×	×	Med	New
PSiTE	✓	✓	✓	✓	✓	✓	×	×	×	×	×	Slow	Low
VarForge	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**Fast**	**Active**

B Coverage Scaling Data

Table 5 reports the full coverage scaling results.

Table 5: Coverage scaling (1 MB reference, 100 mutations, 12 threads, mean of 3 runs). Throughput plateaus at $\sim$6,700 pairs/s; lower than the 10 MB benchmark because the smaller reference provides fewer parallel chunks.

Coverage	Read pairs	Wall (s)	Peak RAM (MB)	Pairs/s
1x	3,333	0.55	10	6,020
5x	16,666	2.58	21	6,467
10x	33,333	5.01	35	6,658
30x	100,000	14.99	91	6,673
50x	166,666	27.17	146	6,135
100x	333,333	53.73	285	6,203
200x	666,666	97.89	563	6,810